Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 3:

Polyalphabetic Ciphers: The Playfair Cipher for the Romanian Alphabet

Elaborated:
Ilie Covali — gr. FAF-243

Verified: asist. univ. Zaica Maia

Chișinău – 2026

Contents

1 Short Theory **2**
1.1 Introduction to Polyalphabetic and Polygraphic Ciphers 2
1.2 Romanian Alphabet Adaptation: The 6×5 Grid Topology 2
1.3 Mathematical Model of the 6×5 Playfair Cipher 3
1.3.1 Digraph Formation and Collision Resolution 3
1.3.2 Encryption Rules 3
1.3.3 Decryption Rules 3
1.3.4 Proof of Invertibility 3
1.4 Key Space and Cryptanalytic Complexity 4

2 Implementation **5**
2.1 Alphabet Definitions and Diacritic Normalization 5
2.2 Input Validation 5
2.3 Matrix Generation 6
2.4 Digraph Formulation 7
2.5 Core Cryptographic Transformation 7

3 Results **9**
3.1 Demonstration 1: Romanian Text Encryption with Diacritics 9
3.2 Demonstration 2: Romanian Text Decryption 9
3.3 Demonstration 3: Standard Cryptographic Sample Execution 11
3.4 Demonstration 4: Defensive Input Validation 13
3.4.1 Keyword Too Short (length < 7) 13
3.4.2 Invalid Characters and Range Suggestion 13
3.4.3 Invalid Operation Choice 13

4 Conclusions **15**

5 Source Code **15**
5.1 Project Repository Structure 15

1 Short Theory

1.1 Introduction to Polyalphabetic and Polygraphic Ciphers

The historical evolution of classical cryptography was largely driven by the inherent vulnerability of monoalphabetic substitution ciphers: the direct preservation of the language's monographic frequency distribution. In simple monoalphabetic substitution, every distinct plaintext character maps deterministically to a fixed ciphertext character throughout the message. Consequently, single-letter frequency analysis, common digram counts, and pattern matching allow an analyst to reconstruct the plaintext and recover the key with modest computational effort.

To overcome this vulnerability, cryptographers developed two fundamental paradigms:

1. **Polyalphabetic Substitution**: Varying the substitution alphabet dynamically as a function of character position. Introduced by Giovan Battista Bellaso in 1553 and later popularized by Blaise de Vigenère in 1586, polyalphabetic ciphers shift characters according to a repeating key phrase, thereby combining multiple shift distributions and flattening the statistical frequency peaks of the plaintext.

2. **Polygraphic (Digraphic) Substitution**: Encrypting units of multiple characters simultaneously rather than individual letters. Invented by Sir Charles Wheatstone in 1854 and championed for military adoption by Lord Lyon Playfair, the *Playfair cipher* treats adjacent letter pairs (digraphs) as atomic encryption blocks. By mapping letter pairs to letter pairs, the cipher diffuses single-letter statistical properties: instead of 26 or 31 possible single-letter outcomes, the analyst must evaluate $26^2 = 676$ or $31^2 = 961$ possible digraph permutations.

1.2 Romanian Alphabet Adaptation: The 6×5 Grid Topology

The standard classical Playfair cipher was originally designed for the 26-letter Latin alphabet. Because a 5×5 square matrix accommodates exactly 25 positions, classical English implementations merge `'I'` and `'J'` into a single cell ($J \to I$), consolidating the 26 letters into 25 discrete elements.

For this laboratory work, the task requires adapting the cipher for the Romanian alphabet comprising $N = 31$ letters:

$$\mathscr{A}_{\text{ro}} = [\text{A, Ă, Â, B, C, D, E, F, G, H, I, Î, J, K, L, M, N, O, P, Q, R, S, Ș, T, Ț, U, V, W, X, Y, Z}] \tag{1}$$

As specified in the laboratory guidelines, when the target alphabet does not form a perfect square, rectangular matrices (6×5, 5×6, etc.) are adopted, and low-frequency or interchangeable letters are consolidated. In the Romanian language, the letter `'J'` occurs with low frequency in core vocabulary and is phonetically and orthographically close to `'I'`. Merging `'J'` into `'I'` ($J \to I$) reduces the 31 letters to exactly 30 unique symbols:

$$\begin{aligned}\mathscr{A}_{\text{playfair}} = [&\text{A, Ă, Â, B, C, D, E, F, G, H, I, Î, K, L, M,} \\ &\text{N, O, P, Q, R, S, Ș, T, Ț, U, V, W, X, Y, Z}]\end{aligned} \tag{2}$$

Because $|\mathscr{A}_{\text{playfair}}| = 30 = 6 \times 5$, all 30 letters populate a rectangular matrix of $R = 6$ rows and $C = 5$ columns without leaving vacant cells.

1.3 Mathematical Model of the 6×5 Playfair Cipher

Let $\mathscr{M}$ denote the 6×5 matrix represented as a bijection:

$$\Phi : \mathbb{Z}_6 \times \mathbb{Z}_5 \to \mathscr{A}_{\text{playfair}}, \quad (r,c) \mapsto \mathscr{M}[r][c] \tag{3}$$

with inverse coordinate lookup $\Phi^{-1} : \mathscr{A}_{\text{playfair}} \to \mathbb{Z}_6 \times \mathbb{Z}_5$, such that $\Phi^{-1}(\alpha) = (r_\alpha, c_\alpha)$.

1.3.1 Digraph Formation and Collision Resolution

A plaintext message M of length $|M|$ is first stripped of whitespace and non-alphabet characters, with all letters converted to uppercase and any occurrence of `'J'` mapped to `'I'`. The string is then partitioned into consecutive digraphs (P_1, P_2):

1. **Collision Rule**: If $P_1 = P_2$ within an active pair, a null separator character is inserted between them. The primary filler character is `'X'` (or `'Z'` if $P_1 =$ `'X'`), deferring P_2 to initiate the subsequent digraph.
2. **Odd-Length Padding**: If an isolated character remains at the end of the text, an additional filler character (`'X'`, or `'Z'` if the terminal letter is `'X'`) is appended to ensure an even number of symbols.

1.3.2 Encryption Rules

For each prepared digraph (P_1, P_2) with matrix coordinates $(r_1, c_1) = \Phi^{-1}(P_1)$ and $(r_2, c_2) = \Phi^{-1}(P_2)$, the ciphertext pair $(C_1, C_2) = E(P_1, P_2)$ is defined as:

$$(C_1, C_2) = \begin{cases} \big(\Phi(r_1, (c_1+1) \bmod 5), \Phi(r_2, (c_2+1) \bmod 5)\big), & \text{if } r_1 = r_2 \text{ (Same Row)} \\ \big(\Phi((r_1+1) \bmod 6, c_1), \Phi((r_2+1) \bmod 6, c_2)\big), & \text{if } c_1 = c_2 \text{ (Same Column)} \\ \big(\Phi(r_1, c_2), \Phi(r_2, c_1)\big), & \text{otherwise (Rectangle Rule)} \end{cases} \tag{4}$$

1.3.3 Decryption Rules

Decryption performs the exact inverse transformation on each ciphertext digraph (C_1, C_2) with coordinates $(r_1, c_1) = \Phi^{-1}(C_1)$ and $(r_2, c_2) = \Phi^{-1}(C_2)$:

$$(P_1, P_2) = D(C_1, C_2) = \begin{cases} \big(\Phi(r_1, (c_1-1) \bmod 5), \Phi(r_2, (c_2-1) \bmod 5)\big), & \text{if } r_1 = r_2 \text{ (Same Row)} \\ \big(\Phi((r_1-1) \bmod 6, c_1), \Phi((r_2-1) \bmod 6, c_2)\big), & \text{if } c_1 = c_2 \text{ (Same Column)} \\ \big(\Phi(r_1, c_2), \Phi(r_2, c_1)\big), & \text{otherwise (Rectangle Rule)} \end{cases} \tag{5}$$

1.3.4 Proof of Invertibility

We verify that $D(E(P_1, P_2)) = (P_1, P_2)$ across all three geometric cases:

- **Case 1 (Same Row, $r_1 = r_2$):**

$$\begin{aligned} c_i' &= (c_i + 1) \bmod 5 \\ c_i'' &= (c_i' - 1) \bmod 5 = ((c_i + 1 \bmod 5) - 1) \bmod 5 = c_i \bmod 5 = c_i \end{aligned} \tag{6}$$

  Since the row index r_i remains unchanged, $\Phi(r_i, c_i'') = \Phi(r_i, c_i) = P_i$.

- **Case 2 (Same Column, $c_1 = c_2$):**

$$r_i' = (r_i + 1) \bmod 6$$
$$r_i'' = (r_i' - 1) \bmod 6 = ((r_i + 1 \bmod 6) - 1) \bmod 6 = r_i \bmod 6 = r_i \tag{7}$$

  Since the column index c_i remains unchanged, $\Phi(r_i'', c_i) = \Phi(r_i, c_i) = P_i$.

- **Case 3 (Rectangle Rule, $r_1 \neq r_2$ and $c_1 \neq c_2$):** Encryption produces $C_1 = \Phi(r_1, c_2)$ and $C_2 = \Phi(r_2, c_1)$. Applying the rectangle rule on (C_1, C_2) yields:

$$P_1' = \Phi(r_1, c_{C_2}) = \Phi(r_1, c_1) = P_1, \quad P_2' = \Phi(r_2, c_{C_1}) = \Phi(r_2, c_2) = P_2 \tag{8}$$

Thus, the transformation is bijective, deterministic, and strictly lossless.

1.4 Key Space and Cryptanalytic Complexity

The theoretical total number of distinct matrix configurations over 30 unique symbols is:

$$N_{\text{total}} = 30! \approx 2.6525 \times 10^{32} \tag{9}$$

However, multiple matrices yield identical digraph substitution mappings due to rotational symmetries. In a 6×5 grid, cyclically shifting all rows preserves vertical relative offsets, and cyclically shifting all columns preserves horizontal relative offsets. There are 6 row shift equivalences and 5 column shift equivalences, yielding $6 \times 5 = 30$ symmetrically equivalent keys for any given matrix.

When the matrix is constructed from a secret keyword K of length ≥ 7, the actual keyspace corresponds to the number of unique character sequences of length m drawn from 30 letters, followed by deterministic alphabetical completion. For a minimal keyword of 7 distinct characters, the partial permutation count is:

$$P(30,7) = \frac{30!}{(30-7)!} = \frac{30!}{23!} = 30 \times 29 \times 28 \times 27 \times 26 \times 25 \times 24 = 10,260,432,000 \approx 1.03 \times 10^{10} \tag{10}$$

For longer keywords ($m \geq 12$), the keyspace exceeds 10^{16} combinations, rendering brute-force exhaustive search computationally prohibitive without specialized parallel architectures.

2 Implementation

The complete cryptosystem is implemented in Python 3 within the module `playfair.py`. It follows defensive programming principles, ensuring thorough input sanitization, clear error reporting, and transparent 2D matrix visualization.

2.1 Alphabet Definitions and Diacritic Normalization

The Romanian alphabet contains five characters with diacritical marks (ă, â, î, ș, ț). To prevent character encoding inconsistencies across different platforms and keyboard layouts, legacy cedilla variants (ş, ţ) are normalized to standard Romanian comma-below characters (ș, ț):

```
ROMANIAN_ALPHABET = [
    "A", "Ă", "Â", "B", "C", "D", "E", "F", "G", "H", "I", "Î", "
       J", "K",
    "L", "M", "N", "O", "P", "Q", "R", "S", "Ş", "T", "Ţ", "U", "
       V", "W",
    "X", "Y", "Z"
]

PLAYFAIR_ALPHABET = [
    "A", "Ă", "Â", "B", "C", "D", "E", "F", "G", "H", "I", "Î", "
       K",
    "L", "M", "N", "O", "P", "Q", "R", "S", "Ş", "T", "Ţ", "U", "
       V", "W",
    "X", "Y", "Z"
]
MATRIX_ROWS = 6
MATRIX_COLS = 5
```

Comment: `PLAYFAIR_ALPHABET` contains exactly 30 characters, perfectly filling the 6×5 matrix after merging $J \to I$.

2.2 Input Validation

Input validation adheres strictly to the constraints outlined in Task 3.1:

1. **Keyword Length**: Must contain at least 7 characters.
2. **Character Set**: Only valid Romanian alphabet letters are accepted. If unsupported characters (digits, symbols, punctuation) are entered, the program halts and informs the user of the allowed character range.

```
def validate_keyword(keyword: str) -> bool:
    if not keyword:
        print("Error: Keyword cannot be empty.")
        return False
    clean_key = keyword.strip()
    if len(clean_key) < 7:
        print(f"Error: Keyword is too short ({len(clean_key)}
           chars)! It must be at least 7 characters long.")
```

```
        return False
    for char in clean_key:
        norm = normalize_character(char)
        if norm not in ROMANIAN_ALPHABET and norm != " ":
            print(f"Error: Invalid character '{char}' in keyword!
               ")
            print("Allowed characters: A-Z, Ă, Â, Î, Ş, Ţ (
               letters only, min length 7).")
            return False
    return True

def validate_text(text: str) -> bool:
    if not text or not text.strip():
        print("Error: Input text cannot be empty.")
        return False
    for char in text:
        norm = normalize_character(char)
        if norm not in ROMANIAN_ALPHABET and norm != " ":
            print(f"Error: Invalid character '{char}' detected!")
            print("Allowed characters: letters from Romanian
               alphabet (A-Z, a-z, ă, â, î, ş, ţ) and spaces.")
            return False
    return True
```

2.3 Matrix Generation

The 6×5 matrix is constructed dynamically from the keyword by deduplicating characters in their order of first appearance and appending unused letters of $\mathscr{A}_{\text{playfair}}$:

```
def build_matrix(keyword: str):
    unique_key = []
    for char in keyword:
        norm = normalize_character(char)
        if norm == " ":
            continue
        c = "I" if norm == "J" else norm
        if c in PLAYFAIR_ALPHABET and c not in unique_key:
            unique_key.append(c)

    for char in PLAYFAIR_ALPHABET:
        if char not in unique_key:
            unique_key.append(char)

    matrix = [unique_key[i:i + MATRIX_COLS] for i in range(0, len
       (unique_key), MATRIX_COLS)]

#     matrix = []
#     for i in range(0, len(unique_key), MATRIX_COLS):
#         row = unique_key[i:i + MATRIX_COLS]
#         matrix.append(row)
    return matrix
```

2.4 Digraph Formulation

Plaintext letters are paired iteratively. When adjacent identical characters are detected within a pair, a filler character is inserted:

```
def prepare_plaintext(text: str):
    clean = []
    for char in text:
        norm = normalize_character(char)
        if norm == " ":
            continue
        c = "I" if norm == "J" else norm
        if c in PLAYFAIR_ALPHABET:
            clean.append(c)

    pairs = []
    i = 0
    n = len(clean)
    while i < n:
        first = clean[i]
        if i + 1 < n:
            second = clean[i + 1]
            if first == second:
                filler = "Z" if first == "X" else "X"
                pairs.append((first, filler))
                i += 1
            else:
                pairs.append((first, second))
                i += 2
        else:
            filler = "Z" if first == "X" else "X"
            pairs.append((first, filler))
            i += 1
    return pairs
```

2.5 Core Cryptographic Transformation

The substitution logic implements modular arithmetic across rows ($c \pm 1 \pmod 5$) and columns ($r \pm 1 \pmod 6$):

```
def encrypt_pair(matrix, pos, p1: str, p2: str):
    r1, c1 = pos[p1]
    r2, c2 = pos[p2]
    if r1 == r2:
        return matrix[r1][(c1 + 1) % MATRIX_COLS] + matrix[r2][(
           c2 + 1) % MATRIX_COLS]
    elif c1 == c2:
        return matrix[(r1 + 1) % MATRIX_ROWS][c1] + matrix[(r2 +
           1) % MATRIX_ROWS][c2]
    else:
        return matrix[r1][c2] + matrix[r2][c1]

```

```
11 def decrypt_pair(matrix, pos, c1: str, c2: str):
12     r1, col1 = pos[c1]
13     r2, col2 = pos[c2]
14     if r1 == r2:
15         return matrix[r1][(col1 - 1) % MATRIX_COLS] + matrix[r2
               ][(col2 - 1) % MATRIX_COLS]
16     elif col1 == col2:
17         return matrix[(r1 - 1) % MATRIX_ROWS][col1] + matrix[(r2
               - 1) % MATRIX_ROWS][col2]
18     else:
19         return matrix[r1][col2] + matrix[r2][col1]
```

3 Results

This section documents actual program execution across Romanian and standard test samples, demonstrating encryption, decryption, matrix generation, and input validation handling.

3.1 Demonstration 1: Romanian Text Encryption with Diacritics

Figure 1 shows the encryption of the Romanian phrase containing diacritics:

`"cifruri polialfabetice și securitatea datelor"`

using the secret keyword `"universitate"`. The program constructs the 6×5 matrix, prints the partitioned digraphs, and outputs the resulting ciphertext:

`"ÎTÂĂRÂTXȘHVTȘLSDIĂTÎĂZNTIFRÂTCĂAVĂKDĂIHȘTȚ"`

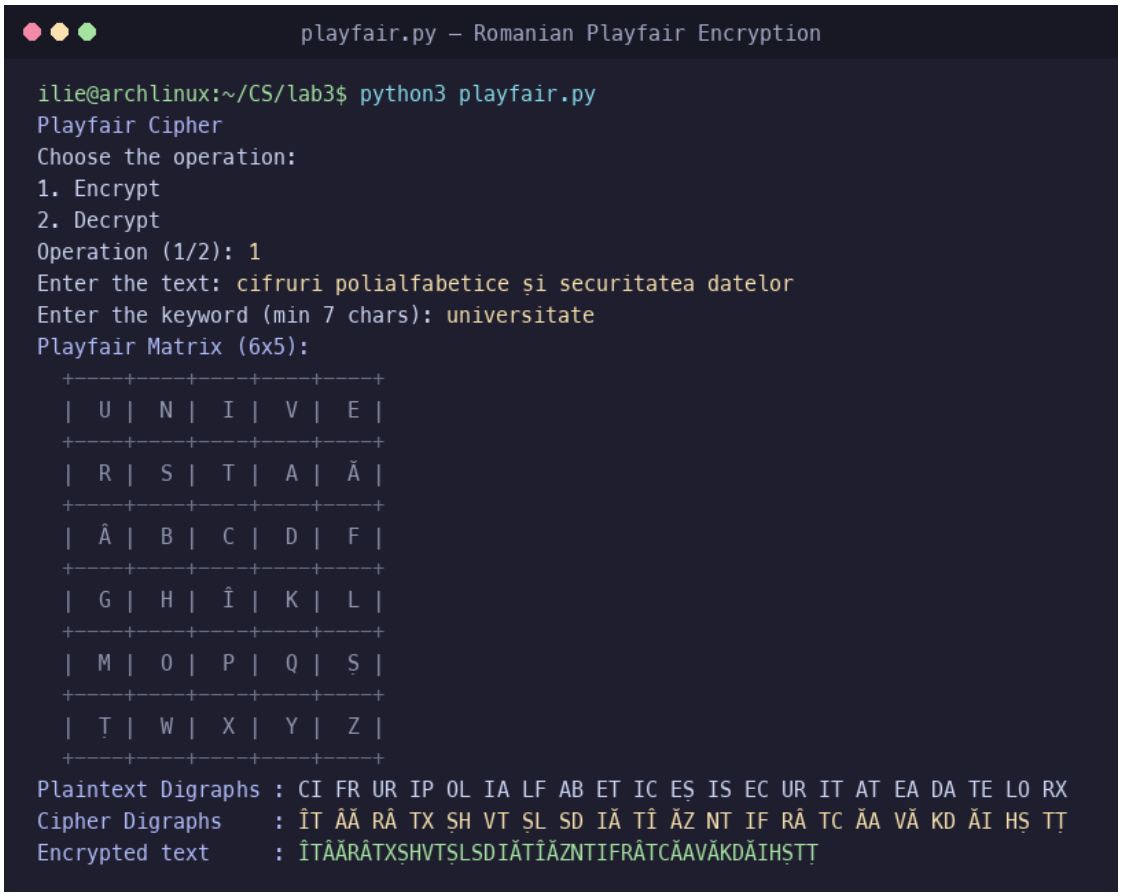


Figure 1: Encryption of Romanian plaintext with diacritics on 6×5 matrix ($K =$ "universitate").

3.2 Demonstration 2: Romanian Text Decryption

Figure 2 demonstrates the exact reverse operation. Decrypting the ciphertext with the same keyword recovers the character stream `"CIFRURIPOLIALFABETICEȘISECURITATEADATELORX"`, which restores the original message `"cifruri polialfabetice și securitatea datelor"` (with trailing padding letter `'X'` trimmed).

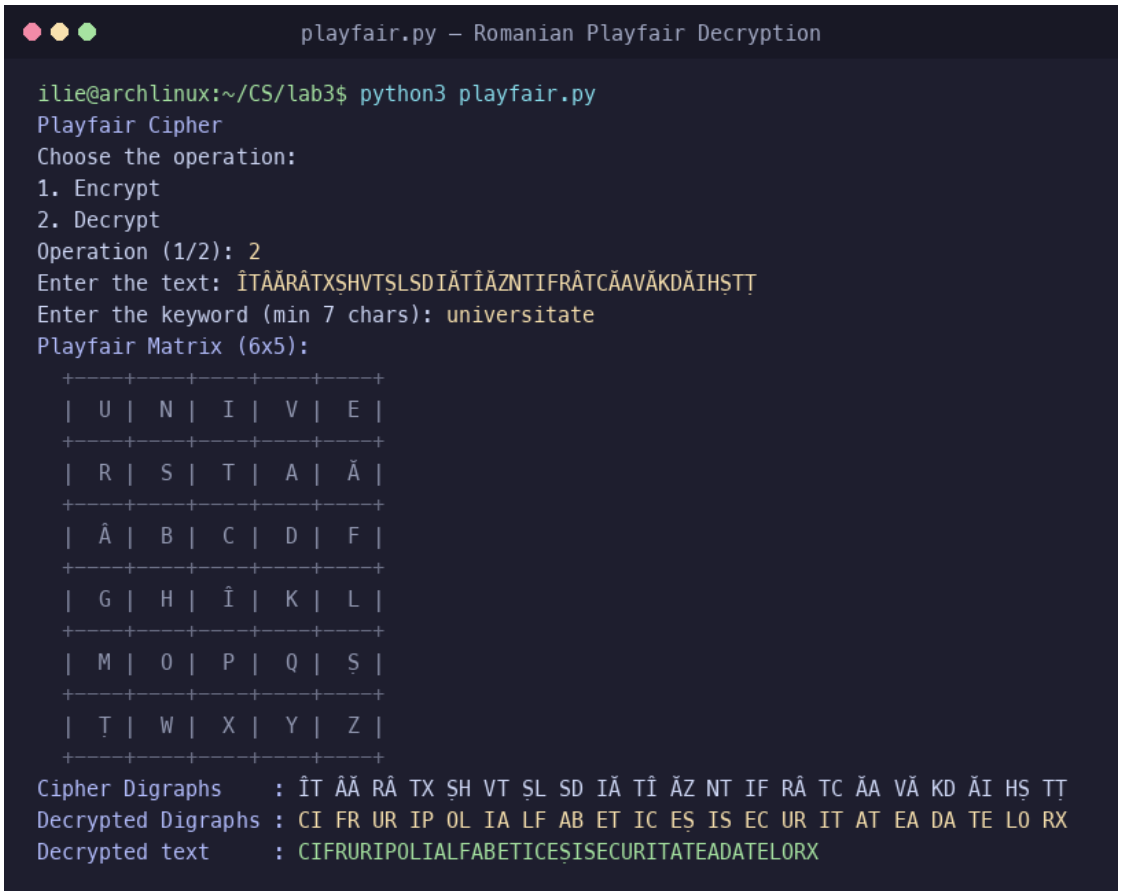


Figure 2: Decryption of Romanian ciphertext on 6×5 matrix ($K =$ ”universitate”).

3.3 Demonstration 3: Standard Cryptographic Sample Execution

Figures 3 and 4 illustrate encryption and decryption of the message `"securitatea informatiei"` using keyword `"criptografie"`. The ciphertext produced is `"UGTQIPIEEHBAMEGCLFCPAT"`, which is then completely decrypted back to its original character sequence.

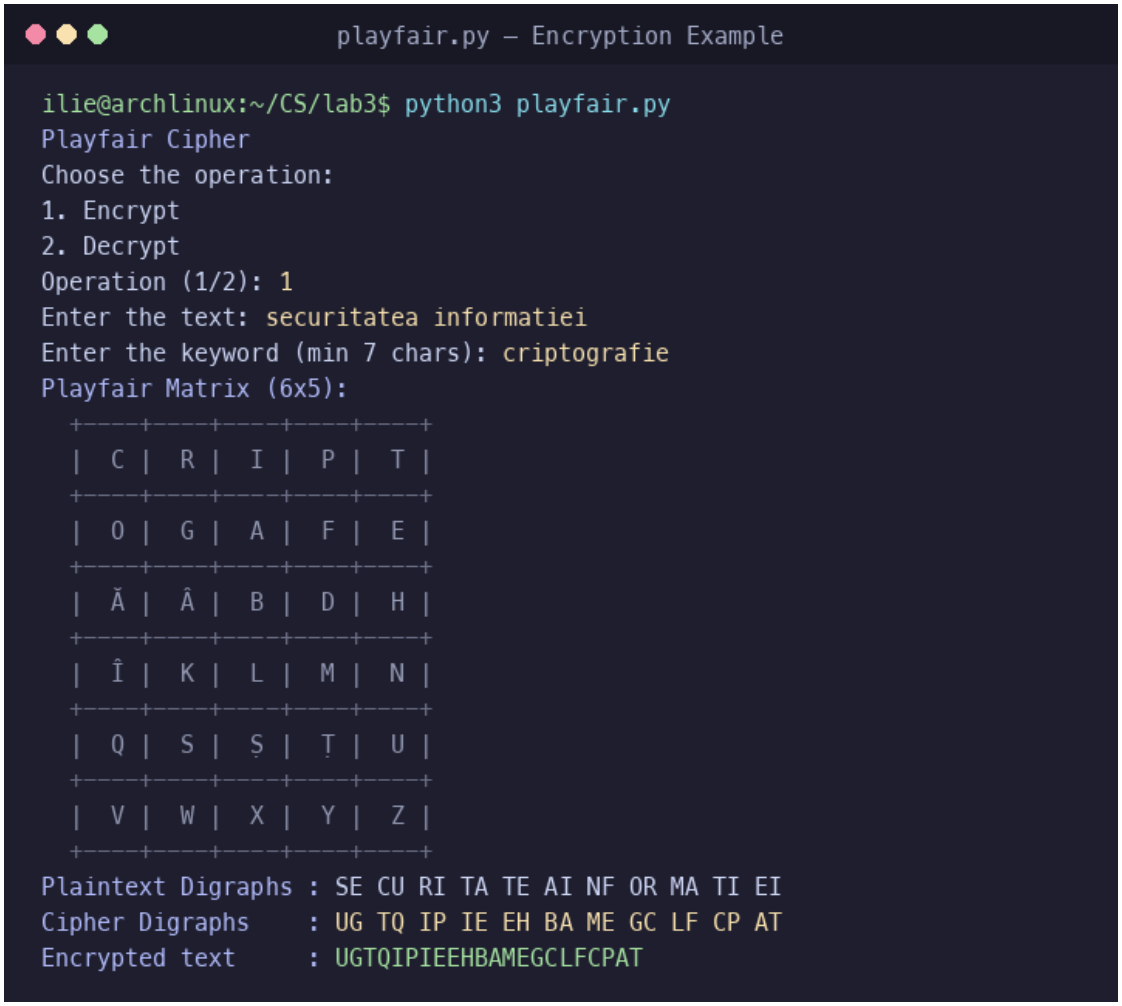


Figure 3: Playfair encryption execution for `"securitatea informatiei"` ($K =$ ”criptografie”).

playfair.py — Decryption Example

```
ilie@archlinux:~/CS/lab3$ python3 playfair.py
Playfair Cipher
Choose the operation:
1. Encrypt
2. Decrypt
Operation (1/2): 2
Enter the text: UGTQIPIEEHBAMEGCLFCPAT
Enter the keyword (min 7 chars): criptografie
Playfair Matrix (6x5):
+----+----+----+----+----+
| C  | R  | I  | P  | T  |
+----+----+----+----+----+
| O  | G  | A  | F  | E  |
+----+----+----+----+----+
| Ă  | Â  | B  | D  | H  |
+----+----+----+----+----+
| Î  | K  | L  | M  | N  |
+----+----+----+----+----+
| Q  | S  | Ș  | Ț  | U  |
+----+----+----+----+----+
| V  | W  | X  | Y  | Z  |
+----+----+----+----+----+
Cipher Digraphs    : UG TQ IP IE EH BA ME GC LF CP AT
Decrypted Digraphs : SE CU RI TA TE AI NF OR MA TI EI
Decrypted text     : SECURITATEAINFORMATIEI
```

Figure 4: Playfair decryption execution for "securitatea informatiei".

3.4 Demonstration 4: Defensive Input Validation

3.4.1 Keyword Too Short (length < 7)

Figure 5 illustrates the defensive rejection of a short keyword (`"cheie"`, length 5). The program detects that the length is below the required 7 characters and halts execution safely with an informative error message.

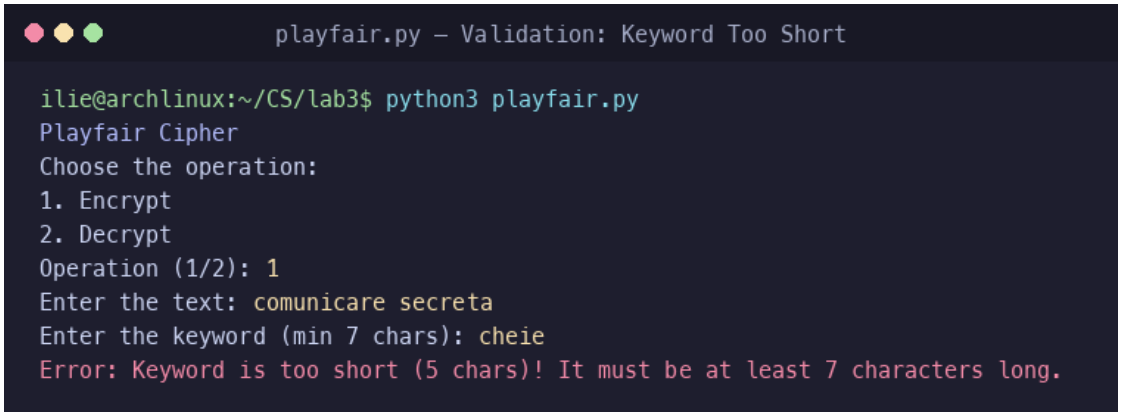


Figure 5: Input validation: Rejection of keyword shorter than 7 characters.

3.4.2 Invalid Characters and Range Suggestion

Figure 6 depicts the detection of illegal characters (underscores, digits, and punctuation in `"parola_secreta#2026!"`). The program alerts the user to the offending character and explicitly specifies the allowed character diapason.

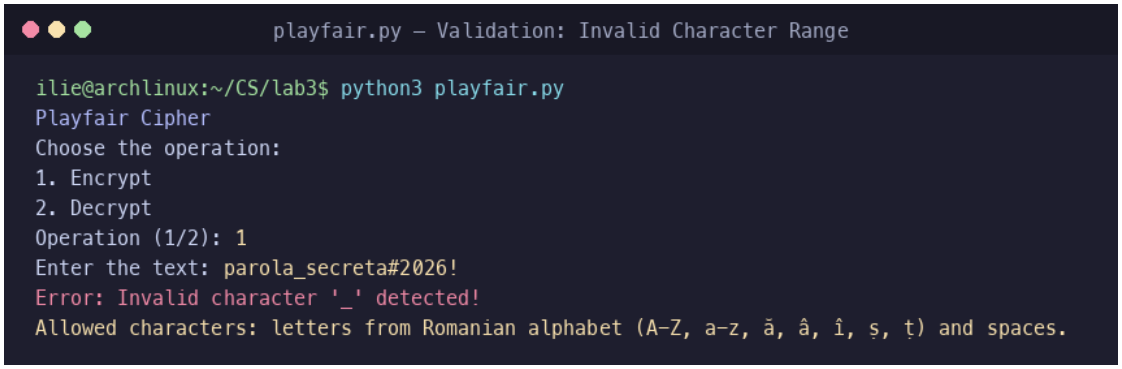


Figure 6: Input validation: Offending character detection and allowed range suggestion.

3.4.3 Invalid Operation Choice

Figure 7 displays the handling of an invalid operation selection (e.g. entering `5` instead of `1` or `2`).

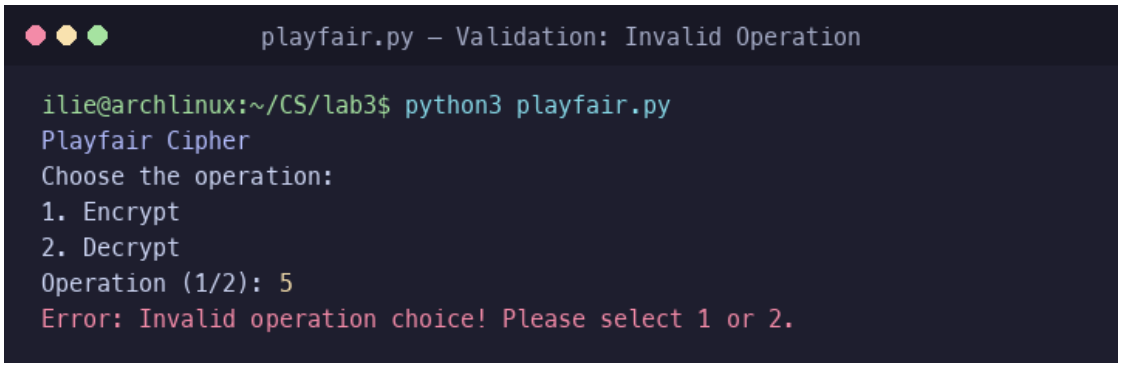


Figure 7: Input validation: Rejection of invalid menu operation choice.

4 Conclusions

Through the implementation, mathematical formalization, and testing of the Playfair cipher adapted for the Romanian alphabet, the following conclusions are established:

1. **Diffusion of Monographic Statistics:** The Playfair cipher represents a major conceptual advance over classical monoalphabetic substitution ciphers (such as standard and permuted Caesar). By operating on digraphs rather than individual characters, single-letter frequency distributions are thoroughly obscured. A single plaintext letter maps to multiple different ciphertext letters depending on its paired counterpart, directly impeding classical monographic frequency cryptanalysis.

2. **Geometrical Flexibility of Rectangular Grids:** While the classical Playfair algorithm relies on a square 5×5 matrix for 25 Latin letters, this laboratory work proved that the underlying geometric substitution rules (same row, same column, rectangle) extend naturally to rectangular matrices of dimension $R \times C$. For the Romanian language ($N = 31$), merging $J \to I$ provides exactly 30 unique symbols that map onto a 6×5 grid without leaving unused cells. The modular arithmetic shifts $(c \pm 1 \bmod 5)$ and $(r \pm 1 \bmod 6)$ maintain bijection and invertibility.

3. **Residual Vulnerabilities of Digraphic Ciphers:** Despite its superiority over Caesar ciphers, Playfair is not unconditionally secure. Because digraphs are treated as atomic units, digraph frequency distributions remain partially preserved. In Romanian, common digraphs such as `"DE"`, `"RE"`, and `"IN"` appear with predictable frequencies. Furthermore, the rectangle rule creates symmetrical pairs (e.g. if $E(AB) = CD$, then $E(BA) = DC$), and the cipher never encrypts a letter to itself. Modern cryptanalysts using simulated annealing, genetic algorithms, or hill climbing can reconstruct the matrix from ciphertexts of a few hundred words.

4. **Deterministic Nature of Known-Key Decryption:** Unlike automated cryptanalysis of Vigenère or Playfair without a key (which generates vast hypothesis spaces requiring language scoring to detect legitimate plaintexts), symmetric decryption with the known secret key is completely deterministic. It yields exactly one unique plaintext character stream, leaving only the standard manual restoration of sentence whitespace as specified by the laboratory guidelines.

5 Source Code

The complete source code for this laboratory work is version-controlled and hosted on GitHub:

GitHub Repository Link:
https://github.com/owhowh0/cs-labs/tree/main/lab3

5.1 Project Repository Structure

- `playfair.py`: Core implementation of the Romanian 6×5 Playfair cipher, defensive validation, and interactive CLI.

- `generate_screenshots.py`: Automation script rendering terminal execution figures.

- `report/`:
  - `report.tex`: The complete LaTeX source code for this report.
  - `report.pdf`: Compiled PDF report.
  - `labreport.cls`: Custom university report document class.
  - `titlesec.sty`: Heading formatting package.
  - `images/`: High-resolution terminal execution captures.
- `pyproject.toml`: Project dependencies and execution metadata managed with `uv`.
- `README.md`: Technical documentation and execution guide.